ФГБОУ ВО «ЮГОРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

ИНЖЕНЕРНАЯ ШКОЛА ЦИФРОВЫХ ТЕХНОЛОГИЙ

МИНИ-ОТЧЁТ

по разработке клиентской части веб-приложения

«Платформа онлайн-хакатонов и соревнований»

Выполнили: студенты группы ПИ41Б, команда № 3

Полуянов, Бойдаченко, Абдурахман, Борисов

Проверил: Исмагулов Милан Ерикович

г. Ханты-Мансийск, 2026 г.

1 ЦЕЛЬ И РЕЗУЛЬТАТ РАБОТЫ

Цель работы — разработать клиентскую часть платформы онлайн-хакатонов, которая взаимодействует с серверным API и отображает данные пользователю в зависимости от его роли.

Создано одностраничное веб-приложение HackPulse в тёмной палитре из оттенков чёрного, синего и красного. Реализованы публичный каталог хакатонов с обратным отсчётом по серверному времени, кабинет участника с командами и invite-кодами, сдача ссылки и архива с индикатором прогресса, кабинет эксперта с оценкой по критериям, кабинет организатора, публичный рейтинг с автообновлением, журнал уведомлений и профиль. Закрыты требования FE-01–FE-05 технического задания.

2 ССЫЛКА НА GITHUB

Исходный код фронтенд-части размещён в репозитории проекта в ветке frontend, каталог frontend/. Адрес каталога:
https://github.com/Andrey787878/hackhaton/tree/frontend/frontend

В ветке также находятся Dockerfile и конфигурация nginx для запуска интерфейса вместе с API командой docker compose up, описание фронтенда docs/FRONTEND.md и скрипт демонстрационных данных scripts/seed_ui_demo.py.

3 ВЫБРАННЫЙ СТЕК ТЕХНОЛОГИЙ

React 18 и TypeScript 5 выбраны для компонентного интерфейса и строгой типизации: ответы API описаны типами, соответствующими схемам Pydantic бэкенда. Маршрутизация страниц и проверка ролей выполнены на React Router 7.

Сборка и dev-сервер — Vite 8: горячая перезагрузка и прокси /api на FastAPI, поэтому браузер работает с одним источником и CORS не требуется. В Docker ту же роль выполняет nginx, который раздаёт собранные файлы и проксирует запросы к контейнеру API.

Стилизация — собственные CSS-файлы с дизайн-токенами без UI-фреймворка: палитра, адаптивная сетка для мобильных устройств, шрифты Inter и JetBrains Mono подключены локально через @fontsource. HTTP-клиент написан на fetch: он добавляет Bearer-токен, однократно обновляет пару токенов при ответе 401 и переводит ошибки сервера на понятный русский текст. Загрузка архива выполняется через XMLHttpRequest ради отображения прогресса. Unit-тесты выполняются в Vitest.

4 СТРУКТУРА ПРОЕКТА

Код разделён на слой сервисов API, состояние авторизации, переиспользуемые хуки и компоненты, страницы и вспомогательные функции. Структура каталогов приведена ниже.

```
frontend/
├── Dockerfile              сборка (Node.js) → раздача (nginx)
├── nginx.conf              SPA, прокси /api → api:8080, лимит 21 МиБ
├── vite.config.ts          dev-прокси /api, настройки Vitest
├── package.json            зависимости и скрипты npm
└── src/
    ├── main.tsx            точка входа: роутер, AuthProvider
    ├── App.tsx             маршруты и защита по ролям
    ├── api/                сервисы работы с REST API
    │   ├── client.ts       fetch, Bearer, refresh, ошибки,
    │   │                   загрузка с прогрессом, скачивание
    │   ├── errors.ts       перевод ошибок FastAPI/Pydantic
    │   ├── types.ts        типы DTO бэкенда
    │   ├── auth.ts         вход, регистрация, профиль
    │   ├── hackathons.ts   хакатоны, кейсы, рейтинг,
    │   │                   кабинет организатора
    │   ├── teams.ts        команды, invite-коды, сдачи
    │   ├── scoring.ts      назначения и оценки экспертов
    │   └── notifications.ts  журнал уведомлений
    ├── auth/               AuthContext, RequireAuth,
    │                       хранение пары JWT
    ├── hooks/              useAsync, useInterval, useServerNow
    ├── components/         Layout, Countdown, CriteriaTable,
    │                       Badges, Feedback, Icon
    ├── pages/              страницы приложения (11 модулей)
    ├── utils/              даты, формула оценки, серверные
    │                       часы, валидация, unit-тесты
    └── styles/             палитра и стили интерфейса
```

Сервисы каталога api/ — единственное место, где формируются HTTP-запросы: страницы вызывают функции вида teamsApi.join(code) и получают типизированный результат или ApiError с сообщением для пользователя. AuthContext хранит текущего пользователя и восстанавливает сессию при

загрузке, RequireAuth закрывает разделы по ролям участника, эксперта и организатора. Компонент Countdown считает время по смещению часов устройства относительно GET /api/v1/time, поэтому таймер соответствует серверному дедлайну.

Ключевые страницы: HomePage — каталог хакатонов; HackathonPage — сроки, кейсы и критерии; TeamPage — состав, управление invite-кодом для капитана и форма сдачи; ExpertPages — пул назначений и оценка; OrganizerHackathonPage — кейсы, критерии, публикация и назначение экспертов; LeaderboardPage — рейтинг. Для страницы итогов на сервер добавлен публичный маршрут GET /api/v1/hackathons/{id}/leaderboard с тестом.

5 СКРИНШОТЫ ИНТЕРФЕЙСА

Скриншоты сделаны в браузере Google Chrome с открытой вкладкой «Сеть» инструментов разработчика (фильтр Fetch/XHR). В журнале видны запросы к API /api/v1 с методами и кодами успешных ответов 200 и 201. Интерфейс запущен в Docker Compose вместе с бэкендом, PostgreSQL и MinIO.

На рисунке 1 показан каталог хакатонов с фильтром по фазе; приложение запрашивает список, серверное время и состояние API.

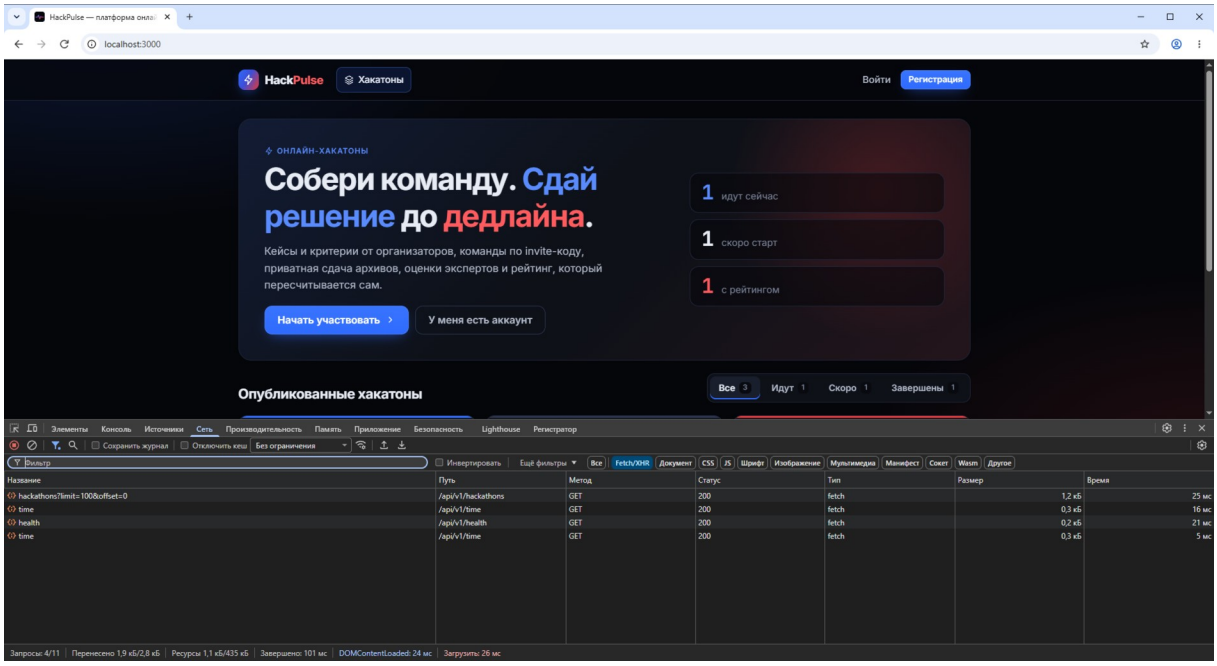


Рисунок 1 – Главная страница: каталог опубликованных хакатонов

На рисунке 2 — страница идущего хакатона: сроки в местном времени и UTC, таймер до дедлайна по серверным часам, кейсы и критерии.

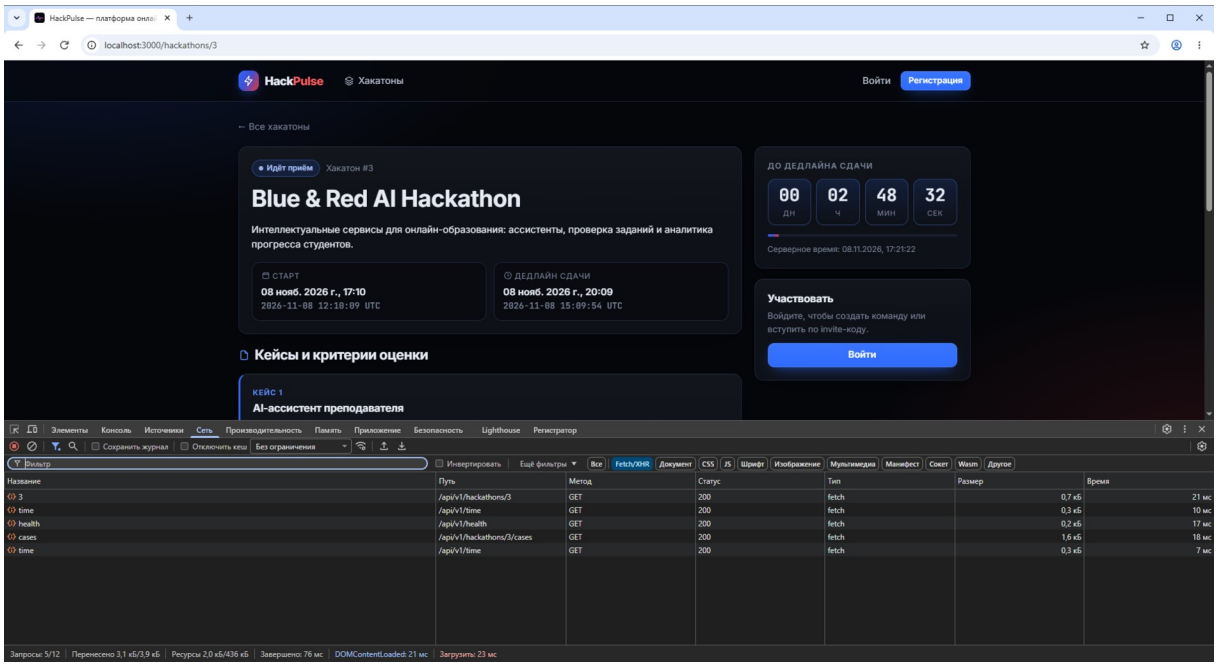


Рисунок 2 – Страница хакатона с обратным отсчётом (FE-01)

На рисунке 3 капитан отправляет ссылку и архив 6 МиБ; виден прогресс загрузки, запрос POST ещё выполняется.

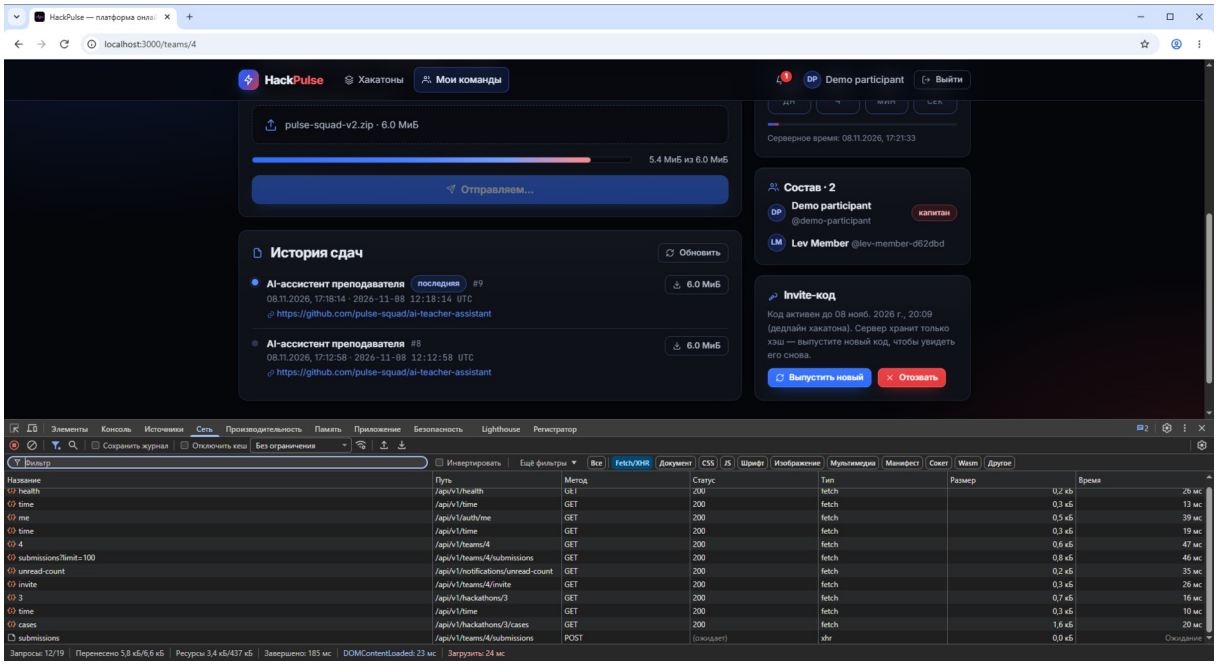


Рисунок 3 – Загрузка архива решения с индикатором прогресса (FE-03)

На рисунке 4 сервер принял решение (POST 201): показаны время сдачи, имя, размер и контрольная сумма архива, обновлена история сдач.

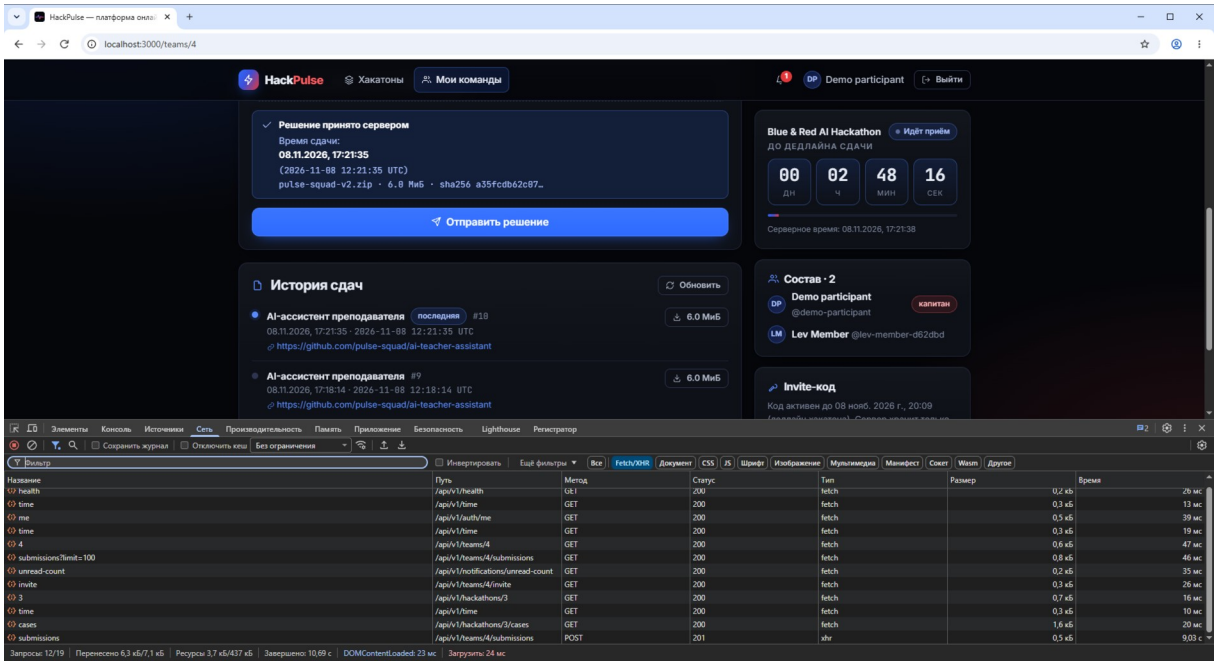


Рисунок 4 – Результат сдачи с серверным временем (FE-03)

На рисунке 5 участник видит состав и историю, но не видит управления invite-кодом и формы сдачи.

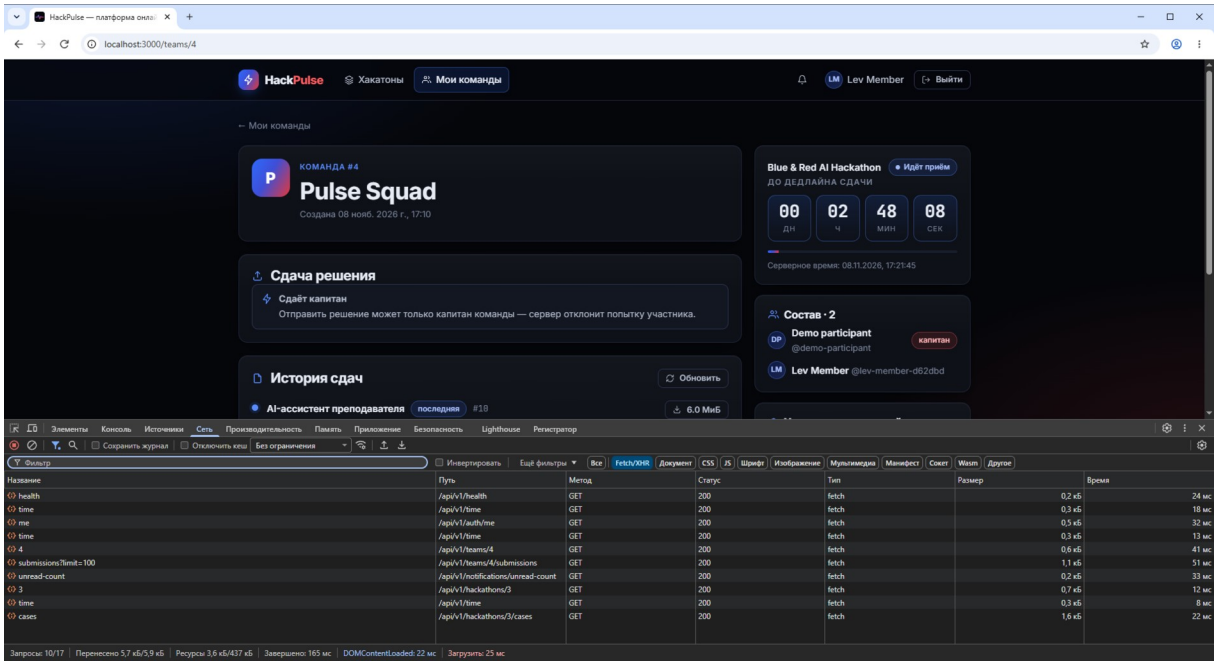


Рисунок 5 – Команда глазами участника, не являющегося капитаном (FE-02)

На рисунке 6 эксперт выставляет баллы с учётом диапазонов и весов; PUT-запрос сохранил оценку, итог 84,00 рассчитан сервером.

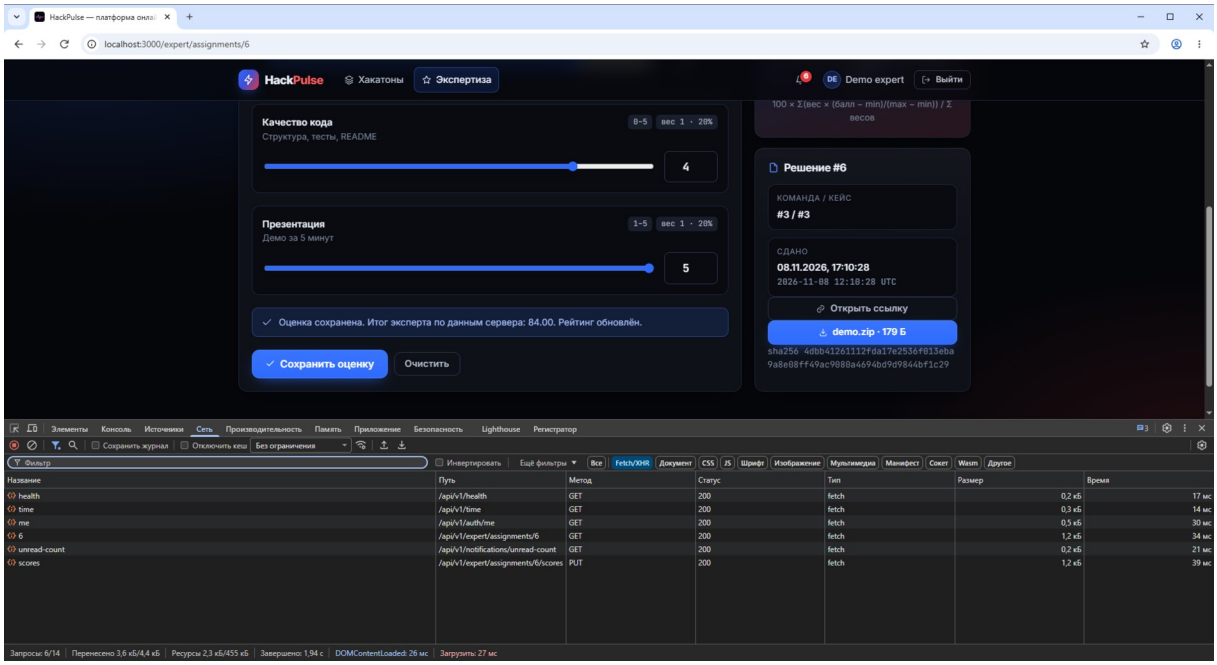


Рисунок 6 – Оценка решения экспертом по критериям (FE-04)

На рисунке 7 показан рейтинг завершённого хакатона по кейсу; данные обновляются каждые 10 секунд.

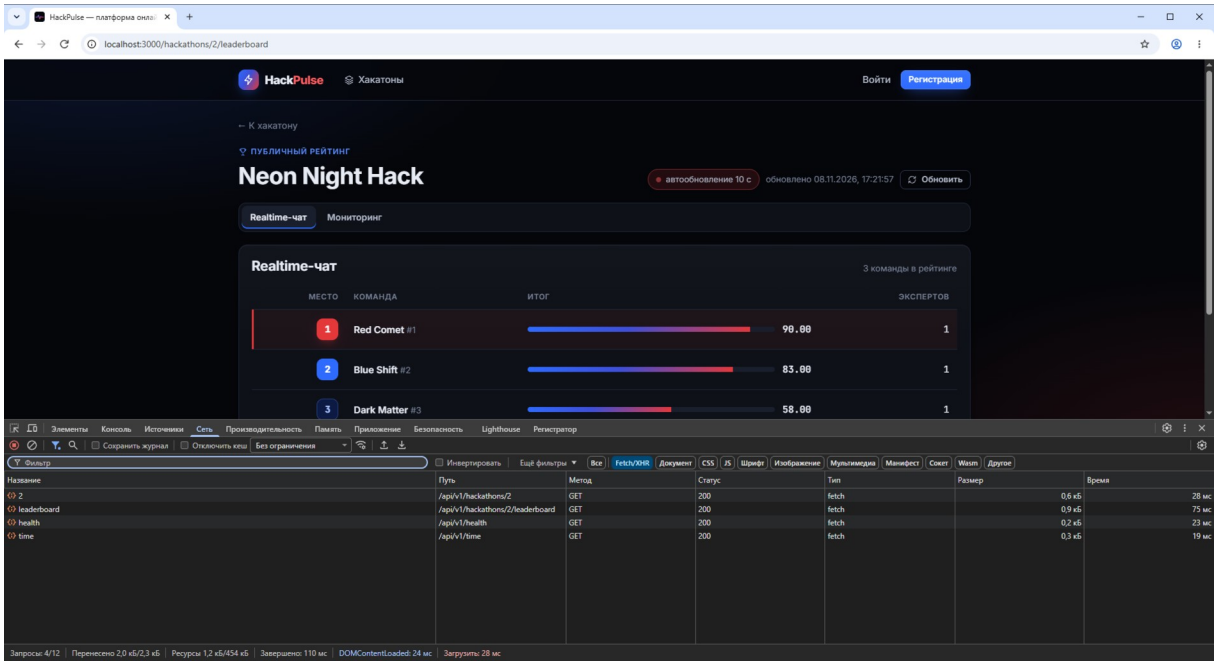


Рисунок 7 – Публичный рейтинг с автообновлением (FE-05)

На рисунке 8 организатор видит команды своего хакатона и назначения экспертов с итоговыми баллами.

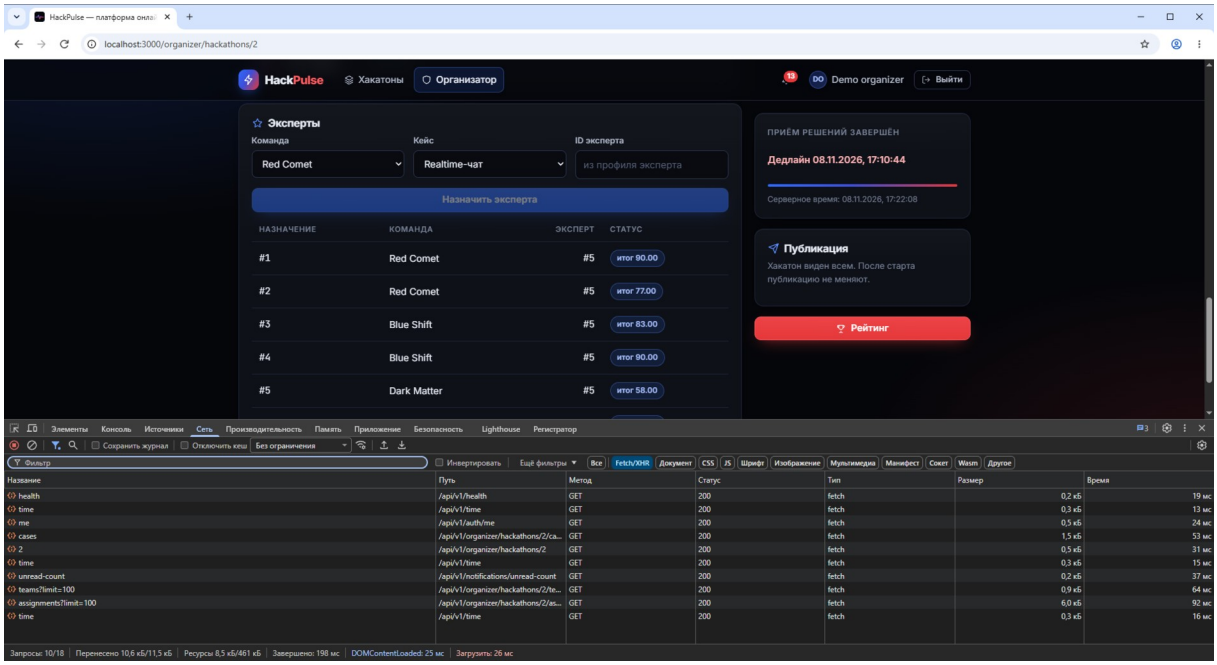


Рисунок 8 – Кабинет организатора: назначение экспертов

На рисунке 9 приведена страница хакатона в мобильном браузере (ширина 390 px): навигация свёрнута в меню, таймер выводится первым.

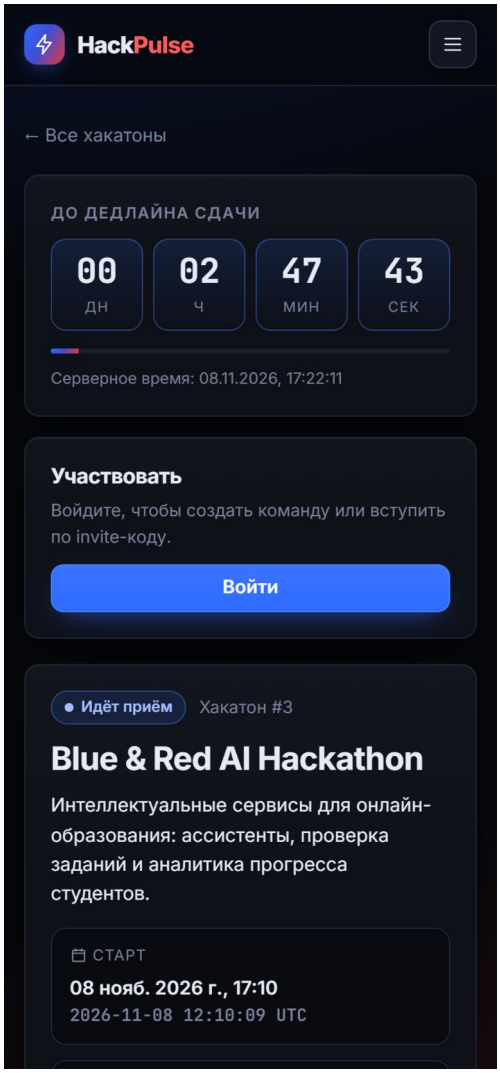


Рисунок 9 – Адаптивная вёрстка на мобильном устройстве

6 ЗАКЛЮЧЕНИЕ

Клиентская часть реализована и связана с серверным API: все данные интерфейс получает и изменяет только через /api/v1, права доступа проверяет сервер. Пройдены сценарии всех ролей: наблюдатель, участник, капитан, эксперт и организатор. Unit-тесты фронтенда (16) и интеграционные тесты бэкенда (98) выполняются успешно; production-сборка запускается в Docker Compose по инструкции README.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Задание 3. Платформа для онлайн-хакатонов и соревнований. URL: https://yusu-source.github.io/GH_pages/03_hackathon.html (дата обращения: 08.11.2026).

2. ГОСТ 7.32—2017. Отчёт о научно-исследовательской работе. Структура и правила оформления.

3. React — документация. URL: https://react.dev (дата обращения: 08.11.2026).

4. Vite — документация. URL: https://vite.dev (дата обращения: 08.11.2026).

5. Репозиторий проекта: https://github.com/Andrey787878/hackhaton.